Experiment No. 6

AIM:
SELECT with various clauses – BETWEEN, IN, Aggregate Functions using Car Parking System.

DESCRIPTION

To view records from the Car Parking System tables using SELECT statements with BETWEEN, IN and Aggregate Functions (AVG, COUNT, SUM, MIN, MAX).

Sample Tables

Vehicle(VehicleID, VehicleNo, OwnerName, VehicleType)
ParkingSlot(SlotID, SlotType, SlotStatus)
ParkingRecord(RecordID, VehicleID, SlotID, HoursParked, Amount)
Payment(PaymentID, RecordID, Amount)

Questions with SQL

- IN: SELECT * FROM Vehicle WHERE VehicleType IN ('Car','Bike');

```
mysql> SELECT * FROM Vehicle;
+-----------+------------+-----------+-------------+
| VehicleID | VehicleNo  | OwnerName | VehicleType |
+-----------+------------+-----------+-------------+
|       101 | TN01AB1234 | Arun      | Car         |
|       102 | TN02CD5678 | Bala      | Bike        |
|       103 | TN03EF9012 | Charan    | Car         |
|       104 | TN04GH3456 | Deepak    | Van         |
|       105 | TN05IJ7890 | Eswar     | Bike        |
+-----------+------------+-----------+-------------+
5 rows in set (0.013 sec)
```

- BETWEEN: SELECT * FROM ParkingRecord WHERE Amount BETWEEN 100 AND 500;

```
mysql> SELECT * FROM ParkingRecord;
+----------+-----------+--------+-------------+--------+
| RecordID | VehicleID | SlotID | HoursParked | Amount |
+----------+-----------+--------+-------------+--------+
|      201 |       101 |      1 |           3 | 150.00 |
|      202 |       102 |      2 |           2 | 100.00 |
|      203 |       103 |      3 |           5 | 250.00 |
|      204 |       104 |      4 |           8 | 400.00 |
|      205 |       105 |      1 |           1 |  50.00 |
+----------+-----------+--------+-------------+--------+
5 rows in set (0.006 sec)
```

- AVG: SELECT AVG(Amount) FROM Payment;

```
mysql> SELECT * FROM Payment;
+-----------+----------+--------+
| PaymentID | RecordID | Amount |
+-----------+----------+--------+
|       301 |      201 | 150.00 |
|       302 |      202 | 100.00 |
|       303 |      203 | 250.00 |
|       304 |      204 | 400.00 |
|       305 |      205 |  50.00 |
+-----------+----------+--------+
5 rows in set (0.008 sec)
```

- MIN MAX: SELECT MIN(Amount), MAX(Amount) FROM Payment;

```
+----------------+----------------+
| Maximum_Amount | Minimum_Amount |
+----------------+----------------+
|         400.00 |          50.00 |
+----------------+----------------+
1 row in set (0.013 sec)
```

- Coursewise: SELECT SlotID, AVG(Amount) FROM ParkingRecord GROUP BY SlotID;

```
+----------------+
| Average_Amount |
+----------------+
|     190.000000 |
+----------------+
1 row in set (0.011 sec)
```

SUM: SELECT SUM(Amount) FROM Payment;

```
mysql> SELECT SUM(Amount) AS Total_Collection
    -> FROM Payment;
+------------------+
| Total_Collection |
+------------------+
|           950.00 |
+------------------+
1 row in set (0.009 sec)
```

- COUNT: SELECT SlotID, COUNT(*) FROM ParkingRecord GROUP BY SlotID;

```
+--------+----------------+----------------+----------------+
| SlotID | Maximum_Amount | Minimum_Amount | Average_Amount |
+--------+----------------+----------------+----------------+
|      1 |         150.00 |          50.00 |     100.000000 |
|      2 |         100.00 |         100.00 |     100.000000 |
|      3 |         250.00 |         250.00 |     250.000000 |
|      4 |         400.00 |         400.00 |     400.000000 |
+--------+----------------+----------------+----------------+
4 rows in set (0.207 sec)
```

- JOIN: SELECT p.SlotType, COUNT(r.RecordID) FROM ParkingSlot p LEFT JOIN ParkingRecord r ON p.SlotID=r.SlotID GROUP BY p.SlotType;

```
+----------+--------------------+
| SlotType | Number_of_Vehicles |
+----------+--------------------+
| Covered  |                  3 |
| Open     |                  2 |
+----------+--------------------+
2 rows in set (0.014 sec)
```

- Vehicle count: SELECT VehicleID, COUNT(*) FROM ParkingRecord GROUP BY VehicleID;

```
+-----------+---------------+
| VehicleID | Total_Parking |
+-----------+---------------+
|       101 |             1 |
|       102 |             1 |
|       103 |             1 |
|       104 |             1 |
|       105 |             1 |
+-----------+---------------+
5 rows in set (0.010 sec)
```

OUTPUTS

Execute each query and paste the corresponding output screenshots.

RESULT

The records were displayed successfully using BETWEEN, IN and Aggregate Functions in the Car Parking System database.